

Three Failure Architectures, Not One: Pattern-Specific Failure Semantics When a Composition Partner Becomes Unavailable in a CKS Deployment

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 7, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to articulate, as a single boundary case with pattern-specific architectural treatments, how a CKS deployment behaves when a composition partner — an adjacent AI component sitting in one of the three composition patterns the source paper supports — becomes unavailable or fails.

Abstract

Real CKS deployments compose with adjacent AI components, and any such partner may at any moment become unavailable or fail. A sibling note formalizes the three positions an adjacent component can occupy relative to a CKS substrate: input to a cell (Pattern A), derived view of substrate content (Pattern B), and separate concern (Pattern C). The present note addresses the question those positions raise under failure: what happens when the partner occupying one of the three positions fails? CKS does not specify a single uniform handling. The three patterns have categorically different failure architectures, and the architectural treatment is pattern-specific. Pattern A consultation failure surfaces inside the consulting cell as a consultation failure governed by the cell's orchestration rule. Pattern B derived-view failure leaves the substrate unaffected because the relationship is unidirectional, with the view recoverable through deterministic regeneration. Pattern C separate-concern failure should leave the substrate unaffected because no coordination coupling exists; any cascade reveals an architectural violation rather than a failure mode the architecture must accommodate. This note formalizes each pattern's failure semantics, identifies six anti-pattern treatments commonly produced under failure pressure, names the operational implications, and gives an operational test of whether a deployment's response to partner failure is CKS-coherent.

1. Why the partner-failure boundary needs to be formalized as standalone

The CKS pattern composes with adjacent AI components through three positions defended at §2.3, §3.1, §4.1, §4.2, §6.1, §6.2, and §11.3 of the source paper, and formalized in a sibling note. That sibling

states what each pattern requires of the substrate—component boundary in the steady state. The present note states what each pattern requires of that boundary *under partner failure*.

The boundary case is operationally common. Composition partners are subject to the ordinary realities of distributed systems: vendor outages, version upgrades that introduce regressions, network partitions, capacity exhaustion, license expirations, decommissioning of legacy services. A deployment using a Pattern A retrieval index, a Pattern B derived knowledge graph, and a Pattern C reference-document service simultaneously is exposed to three independent failure surfaces.

The boundary case is also architecturally non-obvious. The temptation under partner failure is to treat it as a generic "resilience" concern handled by a uniform retry-or-failover layer. That temptation collapses what the architecture has separated: each composition pattern occupies a distinct architectural relationship to the substrate, and partner failure carries a distinct architectural meaning in each pattern. The remedy is to name the failure semantics per pattern.

2. The boundary case, stated precisely

A **composition partner failure** occurs when an adjacent AI component participating in a deployment through Pattern A, Pattern B, or Pattern C becomes unavailable, returns errors, returns degraded output, or otherwise stops fulfilling the contract of its position. The failure may be transient (a brief outage), persistent (a partner that does not return), or partial (some queries succeed, others fail). It may affect one partner or several; in multi-partner deployments, failures may be independent or correlated through shared infrastructure. The deployment continues to operate; the question this boundary case answers is what shape its continued operation takes for each pattern under each partner's failure.

Three architectural commitments are stressed. The composition requirements formalized in a sibling note — per-substrate human governance, conflict preservation across boundaries, addressable provenance across boundaries, AI-as-substrate-mediator at every layer, human-selective composition, plan/trace co-preservation — continue to apply to the partners that remain available; partner failure does not relax any of them. The composition patterns' contracts must specify failure semantics per pattern, since each pattern's relationship to the substrate is categorically different. And the human-selective composition commitment is stressed when partner replacement becomes necessary, because composition membership is architecturally human-authored and cannot be displaced into an automatic, vendor, or framework layer under failure pressure.

3. Pattern A failure: consultation failure governed by the cell's rule

A Pattern A composition partner is consulted by a CKS cell during the cell's execution. The cell reads the substrate as its source of truth, may also query the partner for additional context, and writes its outputs back to the substrate under its orchestration rule. When the partner fails, the consultation fails — from the cell's perspective, structurally similar to any other intra-cell consultation failure (an LLM-call timeout, an API error from a tool the cell uses, a connection error to an external service).

The architectural treatment is that the orchestration rule governing the cell determines the cell's response. The rule may specify retry with bounded attempts, fallback to an alternate consultation source

named in the rule, deferral with the operation queued for later retry, or graceful failure with the cell recording an explicit failure outcome and yielding to human authority. Whichever the rule specifies, cell behavior under failure is rule-governed rather than ad hoc. The provenance of the cell's substrate write — when one occurs — records the consultation that succeeded, the consultations that failed, and the rule disposition that produced the resulting write. Failures are recorded as substrate-addressable provenance rather than vanishing into a runtime log outside the substrate.

If the failure is persistent, the architectural response is partner replacement, and partner replacement is a human authoring action. The orchestration rule changes (or a new rule is authored) to consult an alternate partner. The replacement is exercisable in any environment satisfying the substrate's tool-agnosticism requirements (§7.1), which is what makes alternate-partner replacement feasible without specialized runtime work. The replacement is exercised under human authority, not by automatic logic operating outside the rule.

This pattern's failure semantics overlap with those of LLM consultation failure, treated as its own boundary case in this series, because an LLM consulted by a cell sits in Pattern A relative to the substrate. The overlap is expected: Pattern A is the architectural position for any cell-internal consultation, and the failure semantics derive from the position rather than from the kind of consultation. The two cases formalize the same architecture from different vantages.

4. Pattern B failure: substrate unaffected, view stale or unavailable, regeneration available

A Pattern B composition partner holds a derived view of substrate content — a vector index over substrate text, a knowledge-graph projection, a search-optimized representation. The relationship is unidirectional: substrate content is the source, the view is derived. The view is non-authoritative; coordination questions are answered from the substrate. Pattern B's failure architecture follows directly from this unidirectionality.

When the partner holding the derived view fails, the substrate is unaffected. The substrate did not depend on the view for any answer the substrate is authoritative for; it depended on the view only for the specific query optimization the view supplied. Cells reading the substrate continue to do so. Cells writing to the substrate continue to do so. The view becomes stale (if the partner is reachable but no longer receiving updates) or unavailable (if the partner is unreachable entirely), and operations that depend on the view experience the view as stale or unavailable.

Deterministic regenerability — the property §11.3's source-of-truth commitment implies of any Pattern B partner — is what makes Pattern B's failure architecture closed. The view is, by Pattern B's definition, derivable from substrate state at any time. When the partner returns, the view can be rebuilt from the current substrate; no view-side state was authoritative and so none is irrecoverable. If the partner does not return, the view can be rebuilt in an alternate partner satisfying the same Pattern B contract. The cost of rebuilding scales with substrate size and view-derivation complexity, but the architectural feasibility of rebuilding is unconditional — that is what regenerability commits the architecture to.

Operations that depend on the view query it through cells, and cell behavior under view unavailability is governed by orchestration rules in the same way Pattern A consultation failure is. What the rule must not do — and what an anti-pattern in §6 preempts — is treat a stale or unavailable view as if it were authoritative. The substrate is the source of truth; the view's unavailability is an inconvenience to specific query optimizations, not an authority gap that the view's last known content can fill.

5. Pattern C failure: substrate unaffected, separate concern handles itself, cascade reveals violation

A Pattern C composition partner has no coordination coupling with the substrate. It handles a distinct concern — reference-document queries outside the coordination work, a fine-tuned model used for tasks the substrate does not represent, a service the deployment uses for purposes orthogonal to the cells. By the position's definition, the partner does not write substrate content, does not produce derived views the substrate's cells query, and does not feed into cell consultation under any orchestration rule.

The failure semantics follow directly. When a Pattern C partner fails, the substrate is unaffected because no path exists through which the partner's state could affect substrate state. The separate concern's failure is its own concern, handled by whatever resilience treatment the separate concern's operators apply, which is outside the architecture this paper defends.

The non-obvious case is when a Pattern C partner's failure does appear to affect substrate operation. The architectural reading is that the partner was not in fact occupying Pattern C; some coupling existed that the position should preclude. Common instantiations include shared authentication infrastructure that the substrate also relies on, shared storage that turns out to back both substrate and partner, or shared dependency on a runtime component that surfaces only under failure. The remediation is not to declare Pattern C's failure semantics inadequate; it is to reclassify the partner — into Pattern A or Pattern B, depending on what the coupling actually is — or to remove the coupling so that Pattern C's separation is genuine.

Pattern C's failure semantics are in this sense diagnostic: a claimed Pattern C partner whose failure cascades to the substrate is a hidden-coupling instance, and naming the failure as a Pattern C failure is what makes the hidden coupling visible. The architecture's silence on Pattern C failure modes that affect the substrate is not a gap; it is a constraint, and respecting the constraint is what makes Pattern C separation meaningful.

6. Anti-pattern treatments under partner failure

Six anti-pattern treatments commonly arise when partner failure is handled outside the per-pattern architecture above. Each is a real failure mode that occurs in real deployments; naming each is what makes the drift visible at the moment a practical decision under failure pressure is made.

Composition-partner-failure-corrupts-substrate. A partner's failure causes substrate state to be corrupted, lost, or rolled back. The substrate–cell boundary is breached: the partner's state has reached into the substrate through a path the architecture did not authorize. The fix is to remove the path, not to make the partner more reliable.

Pattern-A-failure-bypasses-rule. A consulting cell, finding its Pattern A partner unavailable, behaves outside the orchestration rule that governs it — substituting an unauthorized alternate source, writing without the rule's authorization conditions met, or proceeding as though the consultation had succeeded. The rule's authority is the architectural authority for cell behavior; bypassing it under failure is bypassing it under any condition.

Pattern-B-stale-view-treated-as-authoritative. A Pattern B view, becoming stale or unavailable, is queried by cells that proceed as though the stale or last-known view were authoritative. The substrate-as-source-of-truth commitment (§11.3) is violated at the moment a coordination question is answered from the view rather than from the substrate.

Pattern-C-failure-cascades-to-substrate. A claimed Pattern C partner's failure affects substrate operation. As §5 develops, the cascade is the architectural signal that the partner was not in fact Pattern C. Treating the cascade as a failure mode the architecture must accommodate is to absorb the violation rather than detect it.

Automatic-partner-replacement. A failed partner is replaced by automatic logic — a framework's default, a vendor's failover, a runtime middleware policy — without an orchestration rule change authored under human authority. Composition membership is architecturally human-authored per the human-selective composition requirement; automatic replacement bypasses that authoring step and turns composition into a property of vendor or framework defaults.

Vendor-managed-failover. A vendor's resilience layer transparently substitutes one partner for another within its own product boundary, presenting the substitution as identity. The substitution may be benign or may be a re-routing across architectural positions (a Pattern A partner failing over into a vendor-managed fallback that occupies a different position relative to the substrate). It is unauthored, undetectable from the cell's vantage, and outside substrate governance — it violates the human-governed commitment as a property of the system's design, even when it happens to behave correctly in any specific instance.

The six share a common shape: an authority that should sit with humans authoring rules over named composition partners has been displaced into an automatic, vendor, or framework layer. Naming the displacement at the architectural level is what allows downstream remediation to address the cause rather than the symptom.

7. Operational implications

Pattern-specific handling is the operational norm, not the exception. A deployment with partners in multiple positions has multiple failure architectures simultaneously, and an operations response that diagnoses which partner failed, identifies the position, and applies the position-appropriate response is doing what the architecture requires. Generic resilience treatments that apply uniform retry-or-failover across all partners flatten the architectural distinctions and tend toward the anti-patterns named in §6.

Partner replacement is rule authoring under human authority. When failure is persistent enough to warrant replacement, replacement is composed by writing or amending the orchestration rules that

name the partner. The labor of replacement is allocable across humans authoring directly, LLMs drafting rule changes under human direction, or stable cells supporting the change — but the authority is not. Tool-agnosticism is what makes replacement feasible without specialized runtime work: the substrate's host environment continues to satisfy the three minimal requirements (§7.1) regardless of which partner currently occupies any composition position.

Partial composition operation continues. A deployment with three composition partners and one failing partner is a deployment with two operational partners and one failed partner. The two operational partners continue to satisfy the composition requirements; the failed partner's position is either being handled per its pattern's failure semantics or being restored through partner replacement. No claim that "the composition has failed" is architecturally warranted because partner failure does not propagate into substrate failure under the per-pattern architecture. The architecture's resilience is in the per-partner localization of failure, not in any partner being inherently reliable.

8. Limits of this boundary case

Several adjacent failure scenarios are formalized as their own boundary cases in this series and do not reduce to this one.

Vendor-level unavailability not at the partner level — a vendor outage that affects multiple partners or services that are not composition partners — is a separate boundary case. A vendor outage that happens to take down a single composition partner is a partner failure for the purpose of this note; a vendor outage that takes down infrastructure shared across the deployment is a different shape and is treated separately.

LLM consultation failure is structurally a Pattern A failure when the LLM is the consultation source, but the LLM-specific case is treated separately because the LLM may sit in any position depending on deployment design and because LLM-side failure modes (rate limiting, content-policy refusals, sampling-induced response-quality degradation) include phenomena that are LLM-specific rather than partner-specific.

Substrate-level failures — the substrate itself unavailable, the substrate's host environment failing, the substrate corrupted by host-level events — are categorically different from composition partner failure. The architectural commitments stressed are different (the substrate-as-source-of-truth commitment, the persistence guarantees of the host environment, the determinism contract), and the treatment is correspondingly different.

Network partitions, deployment-wide capacity exhaustion, and related distributed-systems failures may include partner unavailability among their effects but are not reducible to partner failure and are formalized separately.

The treatment in this note is a per-position localization of partner failure. Treatments the architecture authorizes for partner failure may not be authorized for, and may actively make worse, the adjacent failure cases above; the boundaries between cases matter operationally, not just analytically.

9. Operational test

A deployment's response to composition partner failure is CKS-coherent if and only if all of the following are true at every partner failure event the deployment encounters.

1. The failed partner's position (Pattern A, B, or C) is identifiable, and the response treats the failure per that pattern's semantics rather than uniformly.
2. Substrate state is unaffected by partner failure (Pattern B and Pattern C) or is affected only through the failed partner's consulting cell under its orchestration rule (Pattern A); no path through which partner state corrupts substrate state is permitted.
3. Pattern A consultation failures are handled by the consulting cell's orchestration rule, with the rule's disposition recorded as substrate provenance for any write the cell produces.
4. Pattern B view unavailability does not result in stale or last-known view content being treated as authoritative for coordination questions; the substrate remains the source of truth.
5. A Pattern C partner's failure does not cascade to substrate operation, and any apparent cascade is reclassified as a hidden-coupling violation rather than as a Pattern C failure mode.
6. Persistent partner failure resulting in partner replacement is composed through orchestration-rule changes authored under human authority; no automatic, vendor-managed, or framework-default replacement displaces the human authoring step.

A deployment that fails any of (1)–(6) is responding to partner failure outside the architecture this note formalizes, and the inadequacy is precisely identifiable — the failing item names the architectural commitment that was not preserved.

10. Why naming this boundary case matters

Composition partners are subject to ordinary distributed-systems failure conditions, and a deployment's resilience to such conditions is a real and necessary property. The architectural question this note answers is not whether the deployment should be resilient — it should — but how the architecture authorizes resilience to be implemented without dissolving the distinctions the composition patterns make.

Resilience is implemented per pattern because the patterns are categorically different architectural relationships. A uniform resilience layer on top of all three either flattens the distinctions (and tends toward anti-pattern drift) or installs three pattern-specific behaviors implicitly under one surface (and obscures what is being committed to). The standalone formalization makes the per-pattern semantics explicit and gives downstream implementers a frame in which resilience treatments are evaluable: a treatment is CKS-coherent if it implements per-pattern responses and authors composition changes under human authority, and is not CKS-coherent if it does not.

Earlier boundary cases in this series addressed rule-related, vendor-level, and LLM-consultation boundaries; subsequent ones will formalize authority distribution change, substrate near-capacity behavior, and other edge scenarios where the source paper's architectural commitments meet operational reality. Each names what the architecture commits to in a specific operational shape, so the cumulative set gives downstream implementations a per-shape vocabulary for what the source paper authorizes — and, equally important, what it does not.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Three Failure Architectures, Not One: Pattern-Specific Failure Semantics When a Composition Partner Becomes Unavailable in a CKS Deployment*. May 7, 2026. ORCID: 0009-0004-8065-3235.